

1. For steps, for each letter that is wrong, plus 1. This includes position, so a word with the correct letters but in the wrong positions still adds 1 for each letter. It calculates the lower-bounds of how many moves it will take to reach the goal word. Since every move costs 1 and the heuristic estimates the least number of moves needed to reach the goal word, the heuristic will always be equal to or less than the actual cost to reach the goal state from any state.

For scrabble, for each letter that is wrong, take the correct letter for that position and add its cost. This calculates the lower-bounds of the least cost of letter changes to get the current state word to the goal state word. Similar to the heuristic for steps, it estimates the cost of making every optimal move from the current state to get to the goal state, meaning that every move possible from the current state, even if most optimal, the actual cost will be equal to or less than the estimated cost calculated by the heuristic.

For frequency, for every letter that is wrong, multiply that number by the rarity of the goal state word, and add the number of letters wrong. This includes position. Since the actual cost of moves are the rarity value of the entire word, instead of by letter, in the best case, the current state is simply just 1 letter off and thus the actual cost would just be 1 + the rarity value of the goal state. Knowing that, this heuristic assumes that, the lower-bound being calculated the most optimal least rarity cost, knowing that at the very least, the actual cost will be the goal rarity cost + 1.

2.

Run each problem using all three cost measures. Report:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	6	5	6	0
WHY → ASK	Scrabble	9	6	9	6	0
WHY → ASK	Frequency	38.86	8	38.86	8	0
BEAR → DUCK	Steps	4	5	4	5	0
BEAR → DUCK	Scrabble	11	5	11	5	0
BEAR → DUCK	Frequency	38.24	5	38.24	5	0

- a. UCS and A* always returned the same optimal cost.
 - b. Every cost measure produced the same percentage reduction in expanded nodes.
 - c. Every cost measure was equally informative.
 - d. There were no differences.
3. For steps, yes. The reverse is the same optimal cost. This is due to every step costing only 1 and the length of words being the same. With this, assuming every step is optimal, since the amount of steps to get from the initial to the goal and the goal to the initial is the same, so is the cost.

For scrabble, no. The reverse is not the same since the cost is dependent on the goal state letters. Different words may yield different total costs. If the initial state and the goal state letters total cost in letters were not the same, then cost one way and the other won't be the same since the goal state letters are different. It's possible for the reversible to cost the same, assuming that the total cost of both the initial and goal letters are the same, but that may not always be the case. An example would be why and ask. Assuming the most optimal path, the cost to go from why to ask would be $1 + 1 + 5$, 9. The reverse would be $4 + 4 + 4$, 12.

For frequency, no. The reverse may not be the same since the actual cost does not take into account the rarity of the initial state. With this, if the rarity of the initial state and the goal state are different, then the resulting calculated actual cost will not be the same. If the rarity value of the initial state and the goal state are the same, then the actual cost both ways would be equal, but that may not always be the case. An example would be why and ask. The nodes in between why and ask are the same both ways, resulting in the costs between them resulting in the same values. However, because ask has a rarity value of 4.2326436 and why has a rarity value of 3.1326632, the final value added to the actual cost will differ due to this difference in rarity of the goal state when reversing the words.

4.

a. The model used is Claude's Sonnet 5.

The prompt given:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Show less

The proposed heuristic is

$$h_r(n) = \begin{cases} 0 & n = g \\ (1 + r_{\min}) (h_0(n) - 1) + (1 + \text{rarity}(g)) & n \neq g \end{cases}$$

- b. The precise formula is $h_r(n) = (1 + r_{\min}) * h_0(n) + (\text{rarity}(g) - r_{\min})$.
- c. It is admissible. It assumes the best possible case besides the last move. The best possible case is if the next word's rarity is the minimum rarity value within the entire dictionary. With this, every estimated cost per action is the minimum possible cost possible, thus at most equaling the actual cost and at least, less than. It knows that the last word will eventually be met, assuming it's possible, so it will instead also add the cost of the goal word as well. To summarize, it knows that for every step until before the goal word, the least possible cost is 1 + the lowest rarity value in the dictionary and that the final word's cost is guaranteed.
- d. It's already admissible.

```
DOGCAT(initial=bear, goal=duck, cost=frequency) cost:38.24; time:2.190; solution:bear beak beck buck duck; deltas:[-26.962, -15.513, -7.018, 0.0]
PS C:\Users\andre\Downloads\code_skeleton\code_skeleton> python3 dcsolve.py why ask frequency
DOGCAT(initial=why, goal=ask, cost=frequency) cost:38.86; time:0.977; solution:why way was has his ais ass ask; deltas:[-31.59, -28.653, -27.185, -25.526, -23.559, -9.687, 0.0]
```

- e. PS C:\Users\andre\Downloads\code_skeleton\code_skeleton> █

The cost and number of expanded nodes matched my earlier testing, supporting Claude's proposed heuristic to be admissible and a good heuristic.

AI Disclosure

1. Used Claude Sonnet 5.
2. I used it for debugging and helping me understand some questions on the homework assignment. I also used it to help me come up with the heuristic for frequency.
3. I verified the output of Claude through testing its proposed changes to make sure that it matches the sample outputs, which it did.